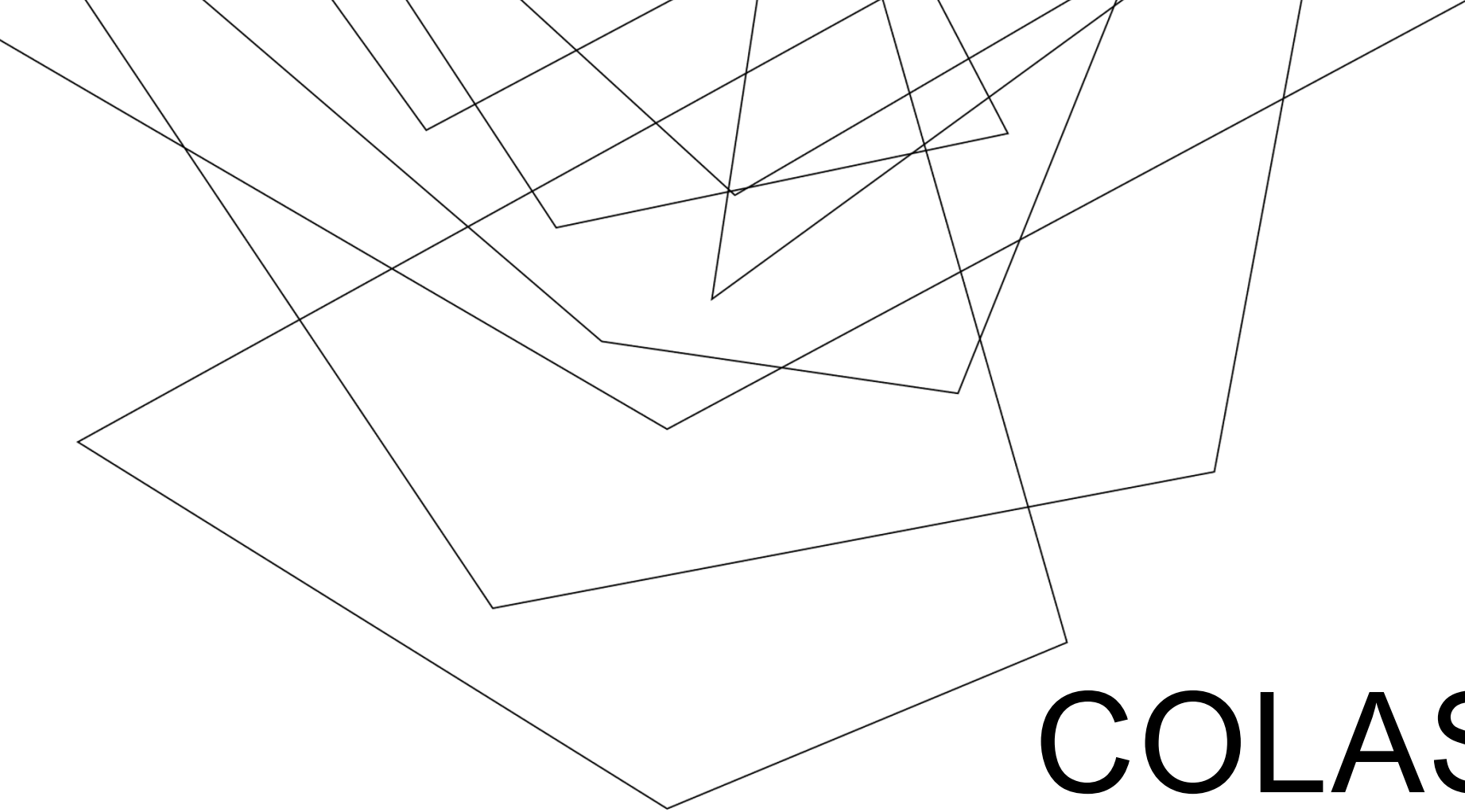




COLAS

Estructura de datos

COLAS

- Es una estructura de datos que almacena elementos en una lista
- Conocidas como FIFO
- Los elementos se insertan por el final
- Los elementos se eliminan por el inicio

Operaciones básicas de una cola:

Crear cola, insertar, quitar, cola vacía, cola llena (excep.) , frente (Inicio), Tamaño de cola.



OPERACIONES

PUSH – Insertar (Encolar)

Se va agregando un dato al final de la cola. El último nodo debe pasar a apuntar al nuevo nodo y el nuevo nodo pasa a ser el nuevo final.

Pop – Borrar (Desencolar)

Se extrae un elemento del frente (inicio) de la cola. Extraer el elemento situado en el inicio de la cola y avanzar el puntero una posición, de esta manera, el nuevo inicio es el segundo elemento

OPERACIONES

Se implementan usando la clase
LinkedList:

Crear (constructor) → Crear cola vacía

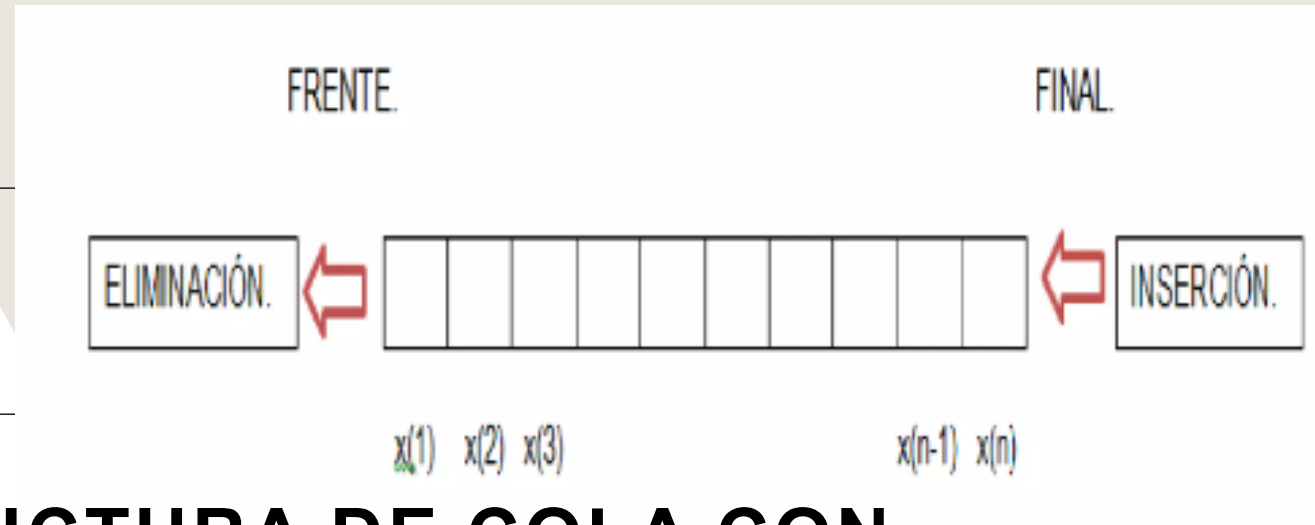
Tamaño (size) → Regresar el número de elementos
de la cola

Adicionar (add u offer) → Añade elemento a la cola
Retirar (poll o remove) → Lee y retira el primer
elemento de la cola.

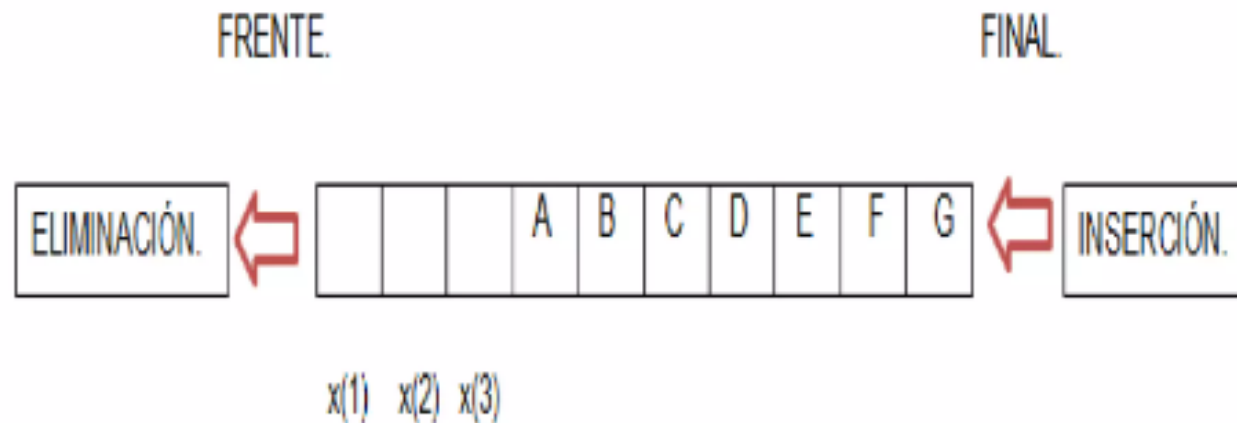
Leer primero (peek) → Lee el primer elemento de la
cola sin retirarlo.

Vacía (isEmpty) → Devuelve true si la cola está sin
elementos o false en caso que contenga algún
elemento.

ESTRUCTURA DE UNA COLA VACÍA



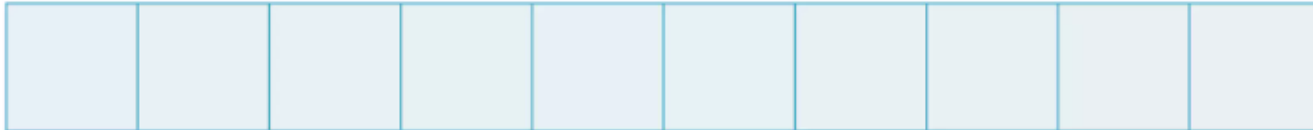
ESTRUCTURA DE COLA CON ELEMENTOS



Crear (constructor):
crea una cola vacía.

```
LinkedList c = new LinkedList();
```

Cola Vacía



Tamaño (*size*): regresa el
número de elementos de la
cola.

```
int n = c.size();
```

n = 0

Leer primero (*peek*):

lee el elemento inicial de la cola sin retirarlo.

```
int x;
```

```
Integer dato = (Integer)c.peek();    x = 92
```

```
x = dato.intValue();
```

Cola c

92	45	65							
----	----	----	--	--	--	--	--	--	--

Leer primero (*poll* o *remove*):
lee el elemento inicial de la cola y
lo retira.

int x;

Integer dato = (Integer)c.poll(); **x = 92**

x = dato.intValue();

Cola c

92	45	65							
----	----	----	--	--	--	--	--	--	--

IMPLEMENTACIÓN CON LISTAS ENLAZADAS – QUEUE - INSERTAR

Algoritmo QUEUE(C: cola, X: ELEMENTO, resp: lógico) es

temp: POSICION;

INICIO

Llena?(C,resp);

si resp = cierto **entonces**

 Escribir “Cola llena”;

 resp := falso;

sino {la cola no está llena, por lo que procedemos a añadir el elemento}

 Obtener(temp);

 temp → .info := X;

 temp → .sgte := nil; {porque va a ser el último}

Vacía?(C,resp);

si resp = cierto **entonces** {será el primero}

 C.prim := temp;

sino C.ult → .sgte := temp; {será el siguiente al último}

finsi;

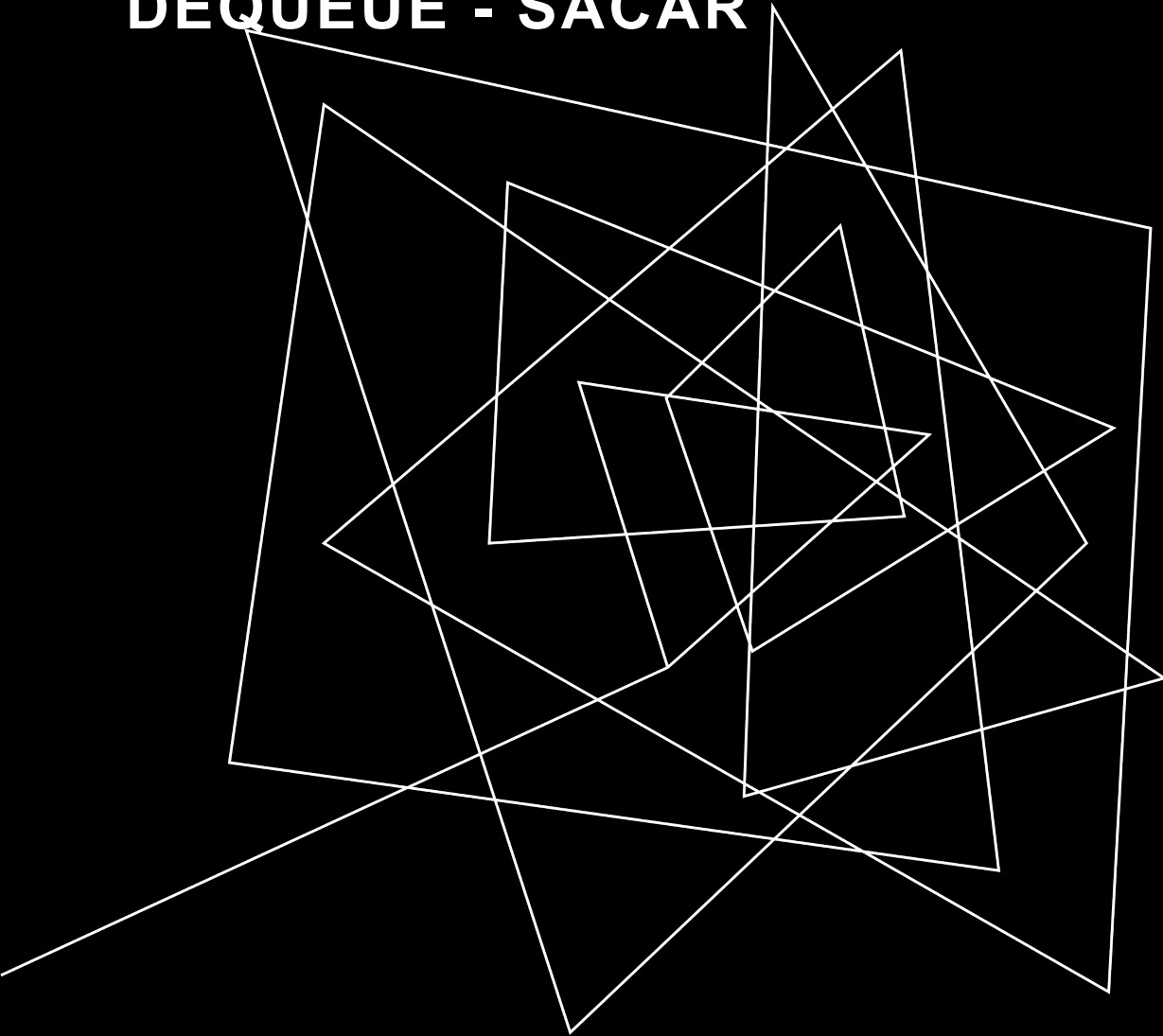
 C.ult := temp; {porque es el nuevo último elemento}

 C.longitud := C.longitud + 1; {pues ahora hay un elemento más en la cola}

finsi;

FIN

IMPLEMENTACIÓN CON LISTAS ENLAZADAS – DEQUEUE - SACAR



Algoritmo DEQUEUE(C: cola, X: ELEMENTO, resp:
lógico) es

temp: POSICION;

tam: numérico;

INICIO

Vacía?(C,resp);

si resp = cierto **entonces**

 Escribir “Cola vacía”;

 resp := falso;

sino

 temp := C.prim;

 E := temp→.info;

Tamaño(C,tam);

si tam = 1 **entonces**

 C.ult := nil;

finsi;

 C.prim := temp→.sgte; {si sólo había un elemento,
 será nil}

 Liberar(temp);

 C.longitud := C.longitud -1;

finsi;

FIN